



Ministério da Educação
Secretaria de Educação Profissional e Tecnológica
Instituto Federal Catarinense
Campus Videira

PAULO SÉRGIO PIERDONÁ

**COMPARAÇÃO DE ESTRUTURAS DE ACELERAÇÃO BVH EM
SIMULAÇÕES DE TEMPO REAL.**

PAULO SÉRGIO PIERDONÁ

**COMPARAÇÃO DE ESTRUTURAS DE ACELERAÇÃO BVH EM
SIMULAÇÕES DE TEMPO REAL.**

Projeto de Trabalho de Conclusão de Curso
apresentado ao Curso de graduação em Ciência da Computação do Instituto Federal Catarinense – Campus Videira para obtenção do título de bacharel em Ciência da Computação.

Orientador: Fábio Pinheiro

Coorientador: professor coorientador

Videira

2026

LISTA DE ILUSTRAÇÕES

Figura 1	– Exemplo do funcionamento do <i>ray casting</i> . Fonte: (WANG, 2022)	9
Figura 2	– Exemplo do funcionamento do Whitted <i>ray casting</i> . Fonte: (WANG, 2022)	10
Figura 3	– Exemplo de triangulo sendo recortado. Fonte: (PINEDA, 1988)	11
Figura 4	– Funções de aresta 3D para um triângulo. (DAVIDOVIČ et al., 2012)	12
Figura 5	– Caso em que primitivas competem pelo mesmo pixel. Criado pelo autor usando GeoGebra Classic.	13
Figura 6	– Imagem path-traced (esquerda). Imagem suavizada (centro). A qualidade de referência renderizada com 2048 amostras por pixel (direita). Fonte (AKENINE-MOLLER; HAINES; HOFFMAN, 2019) cortesia da NVIDIA Corporation.	14
Figura 7	– Exemplo de Bounding Volume Hierarchy (BVH). Fonte: (SINGLETON, s.d.)	16
Figura 8	– Exemplo de travessia de BVH para encontrar a interseção, descartamos toda a subárvore do nó C, uma vez que o raio não intersecta a caixa delimitadora associada a ela. Fonte: (MEISTER et al., 2021)	16

LISTA DE ACRÔNIMOS

AABB	Axis-aligned Bounding Box.
API	Application Programming Interface.
BV	Bounding Volume.
BVH	Bounding Volume Hierarchy.
CPU	Central Processing Unit.
GPU	Graphics Processing Unit.

SUMÁRIO

1	INTRODUÇÃO	5
1.1	Problema de Pesquisa	5
1.2	Objetivos	5
1.2.1	Objetivo Geral	5
1.2.2	Objetivos Específicos	6
1.3	Justificativa	6
2	FUNDAMENTAÇÃO TEÓRICA	7
2.1	Exposição do Tema ou Matéria	7
2.1.1	Hardware dedicado Graphics Processing Units (GPUs)	7
2.1.2	Primitivas	8
2.1.3	Principais efeitos luminosos	8
2.1.4	<i>Ray tracing</i>	8
2.1.5	Rasterização	10
2.1.6	Abordagem híbrida	13
2.1.7	Estruturas de aceleração	15
2.1.8	Métricas	17
3	METODOLOGIA CIENTÍFICA	18
3.1	Orçamento	19
4	CRONOGRAMA	20
	REFERÊNCIAS	21

1 INTRODUÇÃO

A computação gráfica tem evoluído significativamente nas últimas décadas, impulsionada pela necessidade de gerar imagens cada vez mais realistas em diferentes contextos. Nesse cenário, o uso de algoritmos de *ray tracing* tem se destacado pela sua capacidade de produzir imagens altamente realistas. No entanto, o custo computacional associado a essa técnica ainda é um desafio, especialmente em cenários dinâmicos.

Para viabilizar o *ray tracing* em tempo real, torna-se fundamental o uso de estruturas de aceleração. As Bounding Volume Hierarchys (BVHs) permitem reduzir significativamente o número de testes de interseção entre raios e primitivas geométricas. Com o avanço do hardware dedicado, como GPU moderna, diferentes estratégias de construção e atualização dessas estruturas têm sido propostas, cada uma apresentando características distintas em termos de desempenho e eficiência.

Diante desse cenário, torna-se relevante investigar como essas diferentes abordagens se comportam em aplicações de tempo real, considerando tanto as características das cenas quanto as limitações do hardware utilizado.

1.1 PROBLEMA DE PESQUISA

Com o avanço de aplicações em tempo real, como renderização gráfica e simulações interativas, torna-se essencial o uso de estruturas de aceleração eficientes, como as variantes de BVH. No entanto, diferentes algoritmos e estratégias de construção e atualização dessas estruturas, como o rebuild completo e o refit incremental, apresentam comportamentos distintos dependendo do tipo de hardware utilizado e das características da cena.

Diante disso, surge o seguinte problema de pesquisa: qual abordagem de construção e atualização de BVH apresenta melhor desempenho em simulações de tempo real, considerando diferentes cenários de hardware dedicado e cenas dinâmicas?

1.2 OBJETIVOS

1.2.1 Objetivo Geral

Analisar e comparar diferentes estruturas de aceleração baseadas em BVH, avaliando seu desempenho em simulações de tempo real, quando utilizadas em conjunto com hardware dedicado.

1.2.2 Objetivos Específicos

- Analisar as vantagens e desvantagens de diferentes estruturas BVH em simulações de tempo real;
- Avaliar a relevância das estruturas de aceleração em cenários com hardware dedicado;
- Investigar abordagens de construção de BVH, incluindo rebuild completo e refit incremental, em cenas dinâmicas e semi-dinâmicas;
- Avaliar os trade-offs relacionados a:
 - latência por frame;
 - uso de memória;
 - ocupação da GPU e Central Processing Unit (CPU);
 - consumo energético médio;

1.3 JUSTIFICATIVA

A utilização de estruturas de aceleração, como BVH, é fundamental para viabilizar aplicações de alto desempenho em computação gráfica, especialmente em contextos de tempo real. Com a crescente adoção de hardware dedicado, como GPUs modernas e unidades específicas para ray tracing, torna-se necessário compreender como diferentes estratégias de construção e atualização dessas estruturas impactam o desempenho geral do sistema.

Além disso, abordagens como rebuild completo e refit incremental apresentam trade-offs relevantes, que podem influenciar diretamente a eficiência da aplicação dependendo do cenário de uso. A análise desses fatores é importante não apenas do ponto de vista teórico, mas também prático, contribuindo para a escolha de técnicas mais adequadas em aplicações reais.

Dessa forma, este trabalho se justifica pela necessidade de avaliar e comparar essas abordagens, considerando múltiplas métricas de desempenho, a fim de fornecer subsídios para o desenvolvimento de sistemas mais eficientes em ambientes de tempo real.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 EXPOSIÇÃO DO TEMA OU MATÉRIA

2.1.1 Hardware dedicado GPUs

No início, a aparência dos gráficos baseados em GPU era um tanto monótona e padronizada devido à falta de programabilidade, mas agora a maioria dos estágios do pipeline gráfico é programável. Infelizmente, observou-se que o aumento na programabilidade atingiu um limite, e mais liberdade exigiria a capacidade de alterar a própria estrutura do pipeline. Isso, por sua vez, está firmemente enraizado no hardware gráfico específico da GPU e não é facilmente modificado (LAINE; KARRAS, 2011).

Uma grande parte do trabalho no pipeline gráfico, principalmente a execução de shaders de vertex e fragment, mas também outros estágios programáveis, são realizados pelos núcleos de shader programáveis. Entre esses processos destacam-se *marshaling*(organização de dados)) e *scheduling*(agendamento de operações) de dados. Esses processos incluem gerenciamento de filas FIFO, controle de buffer de quadro na memória on-chip, busca de dados de entrada do shader na memória local antes da execução do shader, acionamento de execuções de shader, execução de operações de rasterização. Realiza cálculos mais triviais, como a construção de equações de borda e plano, rasterização e *clipping* (LAINE; KARRAS, 2011).

A NVIDIA introduziu o CUDA em 2007. Por meio do CUDA, os núcleos programáveis (shader cores) passaram a ser acessíveis para aplicações de propósito geral (LAINE; KARRAS, 2011). Tal abordagem possibilita a implementação de algoritmos fora do pipeline gráfico tradicional, ampliando a flexibilidade no desenvolvimento.

A plataforma CUDA apresenta limitações de compatibilidade entre plataformas (SONG et al., 2024). A AMD criou o HIP, que permite aos desenvolvedores migrarem o código CUDA existente para plataformas AMD (Advanced Micro Devices, Inc., 2026). Como alternativa mais multiplataforma que suporta CPUs e outros dispositivos, OpenCL é usado como plataforma de computação paralela (SONG et al., 2024). Um dispositivo OpenCL pode ser dividido em uma ou mais unidades de computação, e cada uma é composta por um ou mais elementos de processamento (PEs) (SONG et al., 2024).

As placas gráficas começaram a oferecer cada vez mais funcionalidades programáveis, como sintaxe para shaders, inconsistência entre fornecedores e GPUs móveis que possuem arquitetura baseada em seus requisitos de energia e espaço. O Vulkan resolve esses problemas

de compatibilidade por ser projetado para arquiteturas gráficas modernas. Vulkan reduz a sobrecarga de lidar com diversos drivers, permitindo que programadores especifiquem suas intenções por meio de uma Application Programming Interface (API) (Vulkan Tutorial, s.d.). Diferentemente das outras plataformas, OpenCL, voltadas à computação de propósito geral, o Vulkan é mais direcionado ao desenvolvimento de aplicações gráficas, como renderização.

2.1.2 Primitivas

Em um sistema de modelagem hierárquica, objetos complexos são construídos a partir da junção de elementos mais básicos organizados em diferentes níveis. Para isso, são necessários elementos que não dependam de outros elementos para serem descritos. Em sistemas hierárquicos, chamamos esses elementos de nível mais baixo de *primitives* (primitivas) (WYVILL, 1995).

No *ray tracing*, as primitivas podem ser: polígonos, objetos geométricos como a esfera ou objetos mais complexos como patches paramétricos (WYVILL, 1995). Na rasterização, é possível usar figuras mais complexas, porém o Vulkan usa apenas pontos, linhas e triângulos, pois são as mais otimizadas. Em especial, os triângulos são amplamente utilizados uma vez que permitem a aproximação de superfícies arbitrárias e complexas (Khronos Group, 2026).

2.1.3 Principais efeitos luminosos

TODO: reflexão; refração; sombras.

2.1.4 Ray tracing

ray tracing é uma técnica de renderização gráfica 3D baseada na simulação da propagação da luz real em um cenário virtual tridimensional. O termo pode ser utilizado tanto para se referir a um algoritmo específico, quanto para uma classe de algoritmos que lançam raios através de uma cena. Comparado com a maioria dos outros algoritmos de renderização, o *ray tracing* proporciona um efeito de luz e sombra mais realista. O primeiro algoritmo de *ray tracing* foi proposto por Appel em 1968. Appel criou o *ray casting* (WANG, 2022).

O *ray casting* renderiza a cena emitindo um raio primário do ponto de observação ou câmera para cada pixel, encontrando o objeto mais próximo que bloqueia o caminho do raio. Raios adicionais, como raios de sombra, podem ser utilizados para verificar a visibilidade da luz a partir do ponto de interseção (WANG, 2022).

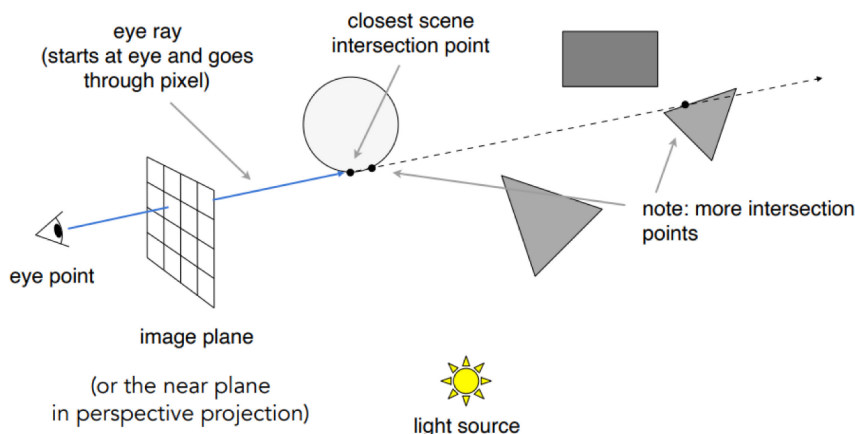


Figura 1 – Exemplo do funcionamento do *ray casting*. Fonte: (WANG, 2022)

As faces são orientadas utilizando a regra da mão direita: os vértices são listados em sentido anti-horário quando se olha o objeto do exterior. Para uma face ABC, o vetor normal $n=AB*AC$ aponta para fora. (FREUD et al., 2006).

A equação do raio que parte da câmera é um dos elementos fundamentais do *ray casting*, pois permite determinar o ponto de interseção com os objetos da cena. Essa equação pode ser expressa como (FREUD et al., 2006):

$$SI = \beta_p SP, \quad \beta_p \in \mathbb{R}, \quad 0 < \beta_p \leq 1$$

Em que:

- S representa a posição de origem do raio;
- P corresponde ao centro de um pixel no detector;
- SP define a direção do raio;
- I é o ponto de interseção do raio e a superfície do objeto;
- β_p é um parâmetro escalar que determina a posição do ponto de interseção ao longo do raio.
- SI é vetor da origem até o ponto de interseção.

O Whitted *ray casting*, uma evolução do *ray casting*, foi desenvolvido por Turner Whitted em 1979. Esse algoritmo usa quatro tipos de raios diferentes. O raio da câmera é similar ao *ray casting*. O segundo raio de reflexão simula superfícies refletoras e semirrefletoras. O terceiro raio de refração pode incidir no objeto a partir de um ângulo. O quarto raio de sombra projeta sombra gerada pela fonte de luz e pelo objeto. A superfície nem sempre é perfeitamente

refletora, seria necessário modelar um componente difuso, mas modelos de iluminação difusa mais realistas implicam maior custo computacional (WANG, 2022).

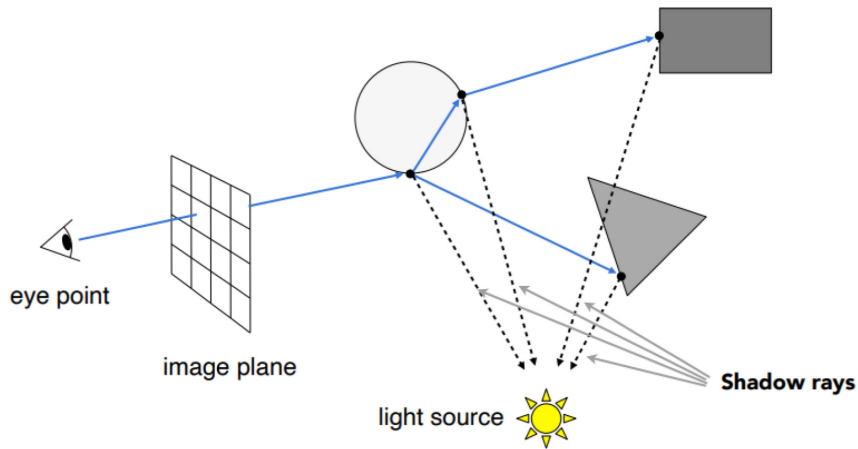


Figura 2 – Exemplo do funcionamento do Whitted *ray casting*. Fonte: (WANG, 2022)

2.1.5 Rasterização

A rasterização é uma técnica básica de renderização. Uma tarefa de renderização complexa pode ser dividida em diferentes subtarefas associadas a diferentes objetos. Em seguida, cada objeto é dividido em polígonos projetados na tela. Finalmente, cada pixel coberto pelos polígonos é renderizado (WANG, 2022). A rasterização era inicialmente feita por software em CPU, isso é praticamente coisa do passado, pois o hardware gráfico dedicado GPU oferece desempenho muito superior e virou o padrão para simulação em tempo real (LAINE; KARRAS, 2011).

O pipeline de rasterização opera em triângulos projetados e recortados em 2D. O coração do algoritmo de rasterização, *coverage computation* (cálculo de cobertura), determina para todos os pixels (e possivelmente várias subamostras em cada pixel), se eles estão cobertos por um determinado triângulo (DAVIDOVIČ et al., 2012).

O teste hierárquico de regiões de pixels para cobertura de triângulos, chamado *binning*, é um fator importante para o desempenho de renderização, pois permite identificar rapidamente áreas da imagem potencialmente cobertas. As regiões podem ser avaliadas em paralelo e hierarquicamente para rapidamente identificar amostras relevantes na tela, reduzindo o número de testes individuais (DAVIDOVIČ et al., 2012).

O *coverage test* (teste de cobertura) normalmente usa funções de aresta 2D lineares no espaço da imagem, uma para cada aresta do triângulo, cujo sinal determina qual lado da aresta está dentro do triângulo. Um pixel está dentro do triângulo se todas as três funções

de aresta forem positivas em sua localização e, portanto, o cálculo da cobertura se torna uma simples avaliação das três funções de aresta, o que é adequado para arquiteturas paralelas (DAVIDOVIČ et al., 2012).

O *clipping* (recorte) à esquerda e à direita pode ser tratado como arestas extras do polígono, representadas por E_l e E_r . Ao considerar esses limites como arestas, o algoritmo de rasterização evita processar partes fora da área visível, parando quando atinge esses limites. O limite superior define onde a renderização começa, e o inferior indica onde ela termina (PINEDA, 1988).

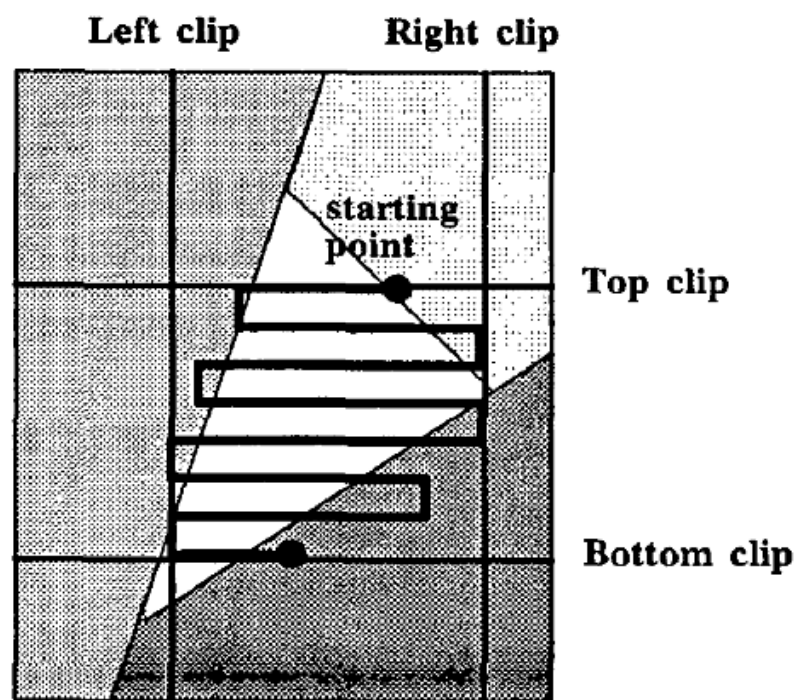


Figura 3 – Exemplo de triângulo sendo recortado. Fonte: (PINEDA, 1988)

Interpolação, qualquer função linear, u , em um triângulo em 3D, por exemplo, cores ou coordenadas de textura, obedece a $u = aX + bY + cZ + d$, sendo $(X, Y, Z)^T$ um ponto em 3D, e os parâmetros a , b e c podem ser determinados a partir de qualquer conjunto de valores definidos nos vértices. Assumindo o espaço canônico do olho, onde o centro de projeção é a origem, a direção da visão é o eixo z e o campo de visão é de 90 graus, dividir esta equação por Z resulta no conhecido esquema de interpolação correto para perspectiva 2D (DAVIDOVIČ et al., 2012).

Testar se uma amostra de pixel é coberta por um triângulo 2D é equivalente a testar se um raio. Um Raio partindo do ponto de vista e passando por esse pixel, intersecta o triângulo. Como exemplo: e é a localização do ponto de vista e o raio passa por $d = d_0 + xdx + ydy$,

para coordenadas de pixel (x, y) , enquanto p_0, p_1, p_2 são os vértices do triângulo. Podemos formular funções de aresta lineares 3D usando o método de volume com sinal, conhecido como teste de interseção raio-triângulo de Plücker. Na prática, usamos $n_2 = (p_1 - p_0) \times (p_2 - p_0)$, que é numericamente mais estável para triângulos pequenos. Definindo a função de aresta 3D correspondente como $V_2(d) = n_2 \cdot d$, que é equivalente a calcular o sinal escalonado (DAVIDOVIČ et al., 2012).

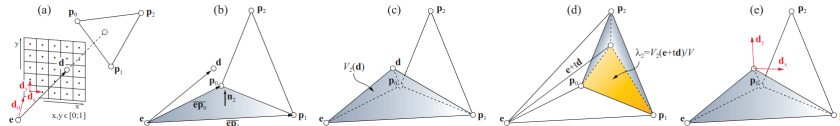


Figura 4 – Funções de aresta 3D para um triângulo. (DAVIDOVIČ et al., 2012)

Grande parte do trabalho no pipeline gráfico, principalmente a execução de shaders de *vertex* (vértice) e *fragment* (fragmento), bem como outros estágios programáveis, é realizada pelos núcleos de shader programáveis (LAINE; KARRAS, 2011).

O *vertex shader* é responsável por decodificar os dados geométricos. Já o *fragment shader* reconstrói o conteúdo da textura ao nível de pixel de saída, por meio da reconstrução do conteúdo da textura via combinação linear (SCHUSTER et al., 2021).

Durante a rasterização, quando uma primitiva intersecta ou passa à frente de outra, torna-se ambíguo qual delas deve ser visível em cada pixel. O Z-buffer é um algoritmo que resolve a visibilidade ao nível de pixel. O Z-buffer Processa os polígonos, atualiza e armazenando os valores de profundidade de cada pixel durante a conversão da varredura. Não lidam bem com superfícies parcialmente transparentes (GREENE; KASS; MILLER, 1993).

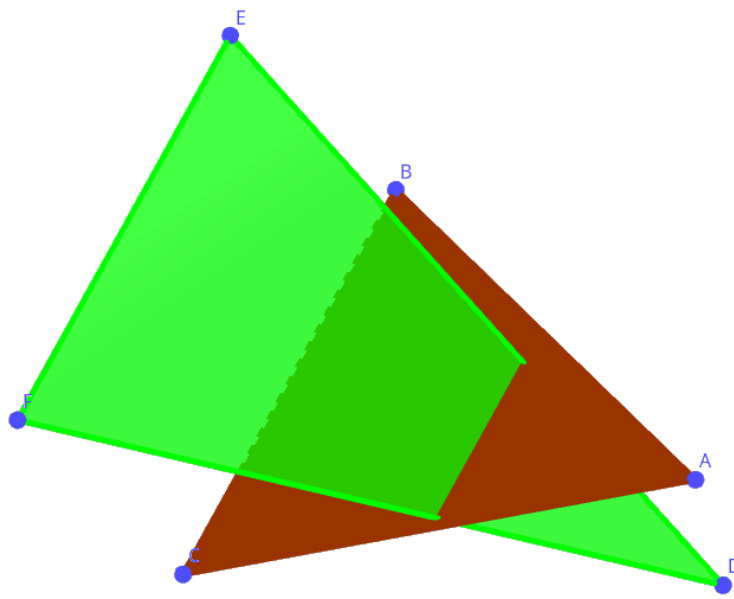


Figura 5 – Caso em que primitivas competem pelo mesmo pixel. Criado pelo autor usando GeoGebra Classic.

A rasterização se tornou a abordagem dominante para aplicações interativas devido aos seus baixos requisitos computacionais iniciais (não requer cálculo de ponto flutuante no espaço de imagem 2D), à sua capacidade de ser implementada em hardware e ao desempenho cada vez maior de placas gráficas dedicadas. No entanto, o tratamento de efeitos globais não é nativo e depende de aproximações, como reflexos e sombras (DAVIDOVIČ et al., 2012).

2.1.6 Abordagem híbrida

Os dois principais algoritmos usados em computação gráfica para gerar imagens 2D a partir de cenas 3D são a rasterização e o *ray tracing*. A rasterização projeta cada triângulo no plano da imagem e enumera todos os pixels cobertos em 2D, enquanto o *ray tracing* opera em 3D, gerando raios através de cada pixel e encontrando a interseção mais próxima com um triângulo (DAVIDOVIČ et al., 2012).

Com a rasterização 3D, as principais diferenças entre as abordagens passam a se concentrar em dois pontos. A travessia da cena, como identificar os objetos relevantes para a cena, usando estruturas e organizações das primitivas. A enumeração de amostras potencialmente cobertas no plano da imagem (binning), que determina quais pixels ou amostras podem ser influenciados por cada primitiva (DAVIDOVIČ et al., 2012).

Rasterização e *ray tracing* são abordagens distintas. Porém, não são técnicas distin-

tas, são pontos extremos de um mesmo espectro contínuo. Sendo assim, diferentes comportamentos podem ser obtidos por meio da ativação ou desativação de determinados componentes, como geração de amostras, binning e testes de oclusão. Permitindo aplicar tanto estratégia de rasterização quanto de ray tracing (DAVIDOVIČ et al., 2012).

Pipeline de renderização híbrida (*Hybrid rendering pipeline*) no qual a rasterização, computação e ray tracing trabalham juntos para possibilitar visuais em tempo real com qualidade próxima à do *ray tracing offline*. A interação de múltiplas etapas gráficas, a utilização das capacidades únicas de cada etapa e a modularização do processo de renderização permitem alcançar vários aspectos visuais de forma otimizada (BARRÉ-BRISEBOIS et al., 2019).

O PICA PICA é um ótimo exemplo. PICA PICA é um experimento de traçado de raios em tempo real com agentes de autoaprendizagem em um mundo gerado proceduralmente. O sistema adota um pipeline de renderização híbrida que combina rasterização, shaders de computação e técnicas de *ray tracing* para compor a imagem final. A arquitetura do pipeline é organizada em múltiplas etapas, cada uma responsável por um aspecto específico da renderização, utiliza uma abordagem modular para maximizar eficiência e qualidade visual (BARRÉ-BRISEBOIS et al., 2019).

Para gerar imagens de alta qualidade, seria necessário gerar milhares de amostras por pixel. Mesmo com uma BVH otimizada, algoritmos eficientes de busca e GPUs rápidas, isso não é possível em tempo real atualmente, exceto para as cenas mais simples. As imagens que podem ser geradas com as restrições de desempenho atuais possuem muito ruído. Porém, elas podem ser tratadas com algoritmos de redução de ruído, podendo gerar imagens com apenas um raio por pixel com qualidade próxima a um *ray tracing offline* (AKENINE-MOLLER; HAINES; HOFFMAN, 2019).



Figura 6 – Imagem path-traced (esquerda). Imagem suavizada (centro). A qualidade de referência renderizada com 2048 amostras por pixel (direita). Fonte (AKENINE-MOLLER; HAINES; HOFFMAN, 2019) cortesia da NVIDIA Corporation.

O pipeline do PICA se inicia com o *object-space rendering* (renderização no espaço de objetos), trata as estruturas geométricas e coordenadas de textura. Faz o cálculo de transparência e translucidez para raios. Depois, com *global illumination*, calcula oclusão (esse processo é limitado a um número máximo de testes por quadro) (BARRÉ-BRISEBOIS et al., 2019). G-buffer, uma abreviação de *geometric buffers* (buffers geométricos), refere-se a buffers usados para armazenar propriedades de superfície. Cada G-buffer é um alvo de renderização separado. Esse alvo pode ser mapa de profundidade, buffer normal, buffer de rugosidade, oclusão, cor da textura (albedo), intensidade da luz, intensidade especular e imagem quase final (AKENINE-MOLLER; HAINES; HOFFMAN, 2019).

Usando dados do G-buffer, gera sombras a partir da fonte de luz e remove possíveis ruídos. Pega pontos de partida do G-buffer para lançar raios de reflexão. Calcula a luz direta que vem de uma fonte até o material. Por fim, pós-processamento que adiciona efeitos como *anti-aliasing* (suavização de serrilhado) e desfoque de movimento (BARRÉ-BRISEBOIS et al., 2019).

2.1.7 Estruturas de aceleração

O *ray tracing* vem se tornando cada vez mais importante para aplicações em tempo real. No entanto, exige alto poder computacional e depende de estruturas de dados pré-processadas para alcançar um desempenho em tempo real. Dentre os métodos para melhorar a eficiência do traçado de raios, as estruturas de dados hierárquicas são extremamente importantes e populares (STICH; FRIEDRICH; DIETRICH, 2009).

Representações hierárquicas de objetos tridimensionais são eficientes tanto em tempo quanto em uso de memória. Essas representações tipicamente consistem em árvores cujos ramos representam volumes delimitadores (RUBIN; WHITTED, 1980). A ideia por trás de uma BVH é subdividir as primitivas de uma cena em conjuntos possivelmente sobrepostos. Para cada um desses conjuntos, um Bounding Volume (BV) é calculado e estes são organizados em uma estrutura de árvore. Os limites de cada nó nesta árvore são escolhidos de forma que ele delimite exatamente todos os nós na subárvore correspondente e cada nó folha delimite exatamente as primitivas contidas (BAUSZAT; EISEMANN; MAGNOR, 2010).

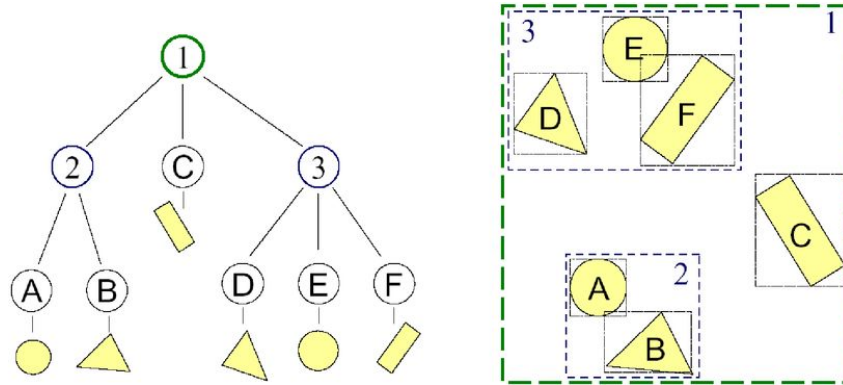


Figura 7 – Exemplo de BVH. Fonte: (SINGLETON, s.d.)

Para encontrar a interseção mais próxima. A travessia de um raio inicia-se no nó raiz e é realizada de forma *top-down* (de cima para baixo). Geralmente, utilizando uma pilha para armazenar nós internos que podem conter a interseção mais próxima. A cada iteração, o nó do topo é removido e o raio é testado contra seu BV. Caso haja interseção, seus nós filhos são adicionados à pilha, no caso de nós internos, ou as primitivas são testadas diretamente, no caso de nós folha. Testando as primitivas, armazenamos a distância até a interseção mais próxima encontrada até o momento. Usando essa distância, podemos ignorar os nós que estão mais distantes do que a interseção encontrada anteriormente. A travessia continua até que a pilha esteja vazia. Para um teste de oclusão, quando queremos saber apenas se o ponto está visível ou ocluído, podemos, portanto, usar um *early exit* (saída antecipada) após encontrar a primeira interseção (MEISTER et al., 2021).

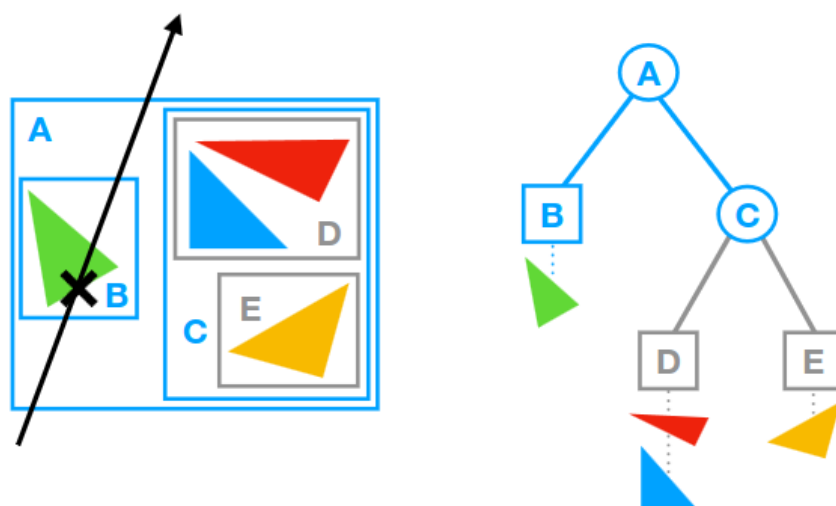


Figura 8 – Exemplo de travessia de BVH para encontrar a interseção, descartamos toda a subárvore do nó C, uma vez que o raio não intersecta a caixa delimitadora associada a ela. Fonte: (MEISTER et al., 2021)

Originalmente, BVH utilizava pelo menos seis *bounding slabs* (planos limitantes)

para formar um casco fechado ao redor de um objeto. Cada slab corresponde à região compreendida entre dois planos paralelos, que restringem o espaço em uma determinada direção; a interseção desses *slabs* define o volume delimitador (BAUSZAT; EISEMANN; MAGNOR, 2010).

Se esses *slabs* são perpendiculares aos eixos do sistema de coordenadas do mundo, o volume delimitador resultante é denominado Axis-aligned Bounding Box (AABB). Em BVHs convencionais, os limites de um AABB são representados por seis valores de ponto flutuante, correspondentes às coordenadas mínimas e máximas ao longo de cada eixo cartesiano (BAUSZAT; EISEMANN; MAGNOR, 2010).

Comparadas a outras abordagens, as BVHs têm geralmente os menores requisitos de memória e um *memory footprint* (uso de memória) pré-computável (BAUSZAT; EISEMANN; MAGNOR, 2010). Técnicas híbridas têm sido investigadas minuciosamente nos últimos anos, buscando combinar os benefícios das árvores BVHs.

2.1.8 Métricas

3 METODOLOGIA CIENTÍFICA

A metodologia em um projeto de TCC descreve o plano detalhado de como a pesquisa será conduzida, incluindo os métodos, técnicas, procedimentos e abordagens que serão utilizados para coletar e analisar dados. Ela é uma parte crucial do projeto, pois fornece uma estrutura sistemática para garantir que os objetivos da pesquisa sejam alcançados de forma eficaz e confiável.

A metodologia inclui várias seções importantes, tais como:

1. Abordagem da pesquisa: Descreve se a pesquisa será qualitativa, quantitativa ou mista. Isso envolve a escolha entre abordagens como estudos de caso, experimentos, levantamentos, entre outros.
2. Instrumentos de coleta de dados: Indica quais instrumentos serão utilizados para coletar dados, como questionários, entrevistas, observações, análise de documentos, entre outros. Essa seção também descreve como esses instrumentos serão desenvolvidos e validados, se necessário.
3. Procedimentos de coleta de dados: Detalha como os dados serão coletados, incluindo informações sobre o local, o período de tempo, os participantes (se houver), os critérios de seleção, entre outros aspectos relevantes.
4. Procedimentos de análise de dados: Descreve como os dados serão processados, analisados e interpretados. Isso pode incluir técnicas estatísticas, análise de conteúdo, codificação de dados, entre outras abordagens.
5. Considerações éticas: Discute as questões éticas relacionadas à pesquisa, como consentimento informado dos participantes, confidencialidade, anonimato, entre outros princípios éticos a serem seguidos durante a condução da pesquisa.

Para trabalhos voltados ao **desenvolvimento de sistemas** é esperado:

1. Levantamento de requisitos
2. Definição de tecnologias/infraestrutura
3. Modelo de Banco
4. Interfaces (prototipação)

3.1 ORÇAMENTO

Se achar necessário, pode citar algo sobre uso ou não de recursos orçamentários

4 CRONOGRAMA

Apresentar croograma de forma visual: tabela ou figura.

REFERÊNCIAS

- Advanced Micro Devices, Inc. **HIP Documentation**. 2026. <<https://rocm.docs.amd.com/projects/HIP/en/latest/index.html>>. Acesso em: 23 abr. 2026.
- AKENINE-MOLLER, Tomas; HAINES, Eric; HOFFMAN, Naty. **Real-time rendering**. [S.l.]: AK Peters/crc Press, 2019.
- BARRÉ-BRISEBOIS, Colin et al. Hybrid rendering for real-time ray tracing. In: **Ray Tracing Gems: High-Quality and Real-Time Rendering with DXR and Other APIs**. [S.l.]: Springer, 2019. p. 437–473.
- BAUSZAT, Pablo; EISEMANN, Martin; MAGNOR, Marcus A. The minimal bounding volume hierarchy. In: **VMV**. [S.l.: s.n.], 2010. p. 227–234.
- DAVIDOVIČ, Tomáš et al. 3d rasterization: a bridge between rasterization and ray casting. In: **Proceedings of graphics interface 2012**. [S.l.: s.n.], 2012. p. 201–208.
- FREUD, Nicolas et al. Fast and robust ray casting algorithms for virtual x-ray imaging. **Nuclear Instruments and Methods in Physics Research Section B: Beam Interactions with Materials and Atoms**, Elsevier, v. 248, n. 1, p. 175–180, 2006.
- GREENE, Ned; KASS, Michael; MILLER, Gavin. Hierarchical z-buffer visibility. In: **Proceedings of the 20th annual conference on Computer graphics and interactive techniques**. [S.l.: s.n.], 1993. p. 231–238.
- Khronos Group. **Vulkan Documentation**. 2026. <<https://docs.vulkan.org/spec/latest/index.html>>. Acesso em: 23 abr. 2026.
- LAINE, Samuli; KARRAS, Tero. High-performance software rasterization on gpus. In: **proceedings of the acm siggraph symposium on high performance graphics**. [S.l.: s.n.], 2011. p. 79–88.
- MEISTER, Daniel et al. A survey on bounding volume hierarchies for ray tracing. In: WILEY ONLINE LIBRARY. **Computer Graphics Forum**. [S.l.], 2021. v. 40, n. 2, p. 683–712.
- PINEDA, Juan. A parallel algorithm for polygon rasterization. In: **Proceedings of the 15th annual conference on Computer graphics and interactive techniques**. [S.l.: s.n.], 1988. p. 17–20.
- RUBIN, Steven M; WHITTED, Turner. A 3-dimensional representation for fast rendering of complex scenes. In: **Proceedings of the 7th annual conference on Computer graphics and interactive techniques**. [S.l.: s.n.], 1980. p. 110–116.
- SCHUSTER, Kersten et al. Compression and rendering of textured point clouds via sparse coding. In: **High Performance Graphics (Symposium Papers)**. [S.l.: s.n.], 2021.
- SINGLETON, Stanley. **Acceleration Structures for Ray Tracing**. s.d. Acesso em: 15 abr. 2026. Disponível em: <<https://slideplayer.com/slide/14175246/>>.
- SONG, Yupu et al. A parallel canny edge detection algorithm based on opencl acceleration. **Plos one**, Public Library of Science San Francisco, CA USA, v. 19, n. 1, p. e0292345, 2024.

STICH, Martin; FRIEDRICH, Heiko; DIETRICH, Andreas. Spatial splits in bounding volume hierarchies. In: **Proceedings of the Conference on High Performance Graphics 2009**. [S.l.: s.n.], 2009. p. 7–13.

Vulkan Tutorial. **Vulkan Tutorial**. s.d. <<https://vulkan-tutorial.com/Overview>>. Acesso em: 23 abr. 2026.

WANG, Zijin. The development of ray tracing and its future. In: **CONF-SPML**. [S.l.: s.n.], 2022. p. 19–28.

WYVILL, Geoff. Practical ray tracing. In: **Computer Graphics International**. [S.l.: s.n.], 1995.